

07 — Shared Memory e Bank Conflicts

Pré-requisito: [04 — memória](#) e [06 — occupancy](#).

1. Shared memory como scratchpad cooperativo

Threads do mesmo block podem carregar dados globais para shared memory, sincronizar e reusar esses dados muitas vezes. Diferente de cache, o programa escolhe endereços e momento da cópia.

```
VRAM → carga cooperativa → shared → muitos usos → resultado
```

Isso aumenta arithmetic intensity quando evita releituras globais. Também custa instruções, barreiras e capacidade que poderia manter mais blocks residentes.

2. Banks

Shared memory é dividida em banks para servir múltiplas lanes simultaneamente. Em uma configuração comum, palavras consecutivas de 32 bits mapeiam para banks consecutivos:

```
bank ≈ (endereço_em_palavras) mod 32
```

Se lanes de uma instrução acessam banks distintos, o acesso pode ocorrer em paralelo. Se múltiplas lanes pedem endereços diferentes no mesmo bank, o acesso pode ser dividido em etapas: **bank conflict**. O grau do conflito indica quantas parcelas serializadas são necessárias.

Broadcast — lanes lendo o mesmo endereço — é tratado de forma especial e não deve ser confundido com conflito de endereços diferentes.

3. Padding como transformação de layout

Uma matriz shared `tile[32][32]` acessada por coluna pode fazer várias lanes caírem no mesmo bank. Declarar `tile[32][33]` desloca o início das linhas e pode distribuir os banks. O byte extra não é magia: muda a função endereço → bank.

Pergunta rápida

Por que padding pode melhorar shared memory e ao mesmo tempo reduzir occupancy?

► Resposta

O que guardar

- Shared é explícita, por block e exige coordenação.
- Bank conflict é sobre acessos simultâneos de um warp a endereços diferentes no mesmo bank.
- Tiling só vale quando reuso paga cópia, sincronização e pressão de recursos.

Conexão: mesmo um layout perfeito espera VRAM em algum momento; o próximo arquivo explica como a GPU esconde essa latência.